

CHAPTER 6: IMPLEMENTATION & TECHNOLOGY STACK

Detailed Technology Justifications, Implementation Pipeline & GraphRAG Mechanics

6.1 Introduction

The implementation of ResearchGraph AI follows a modular architecture where each technology is responsible for a specific functionality within the system. The frontend provides an interactive interface for researchers, while the backend manages document processing, semantic retrieval, graph operations, and communication with Large Language Models.

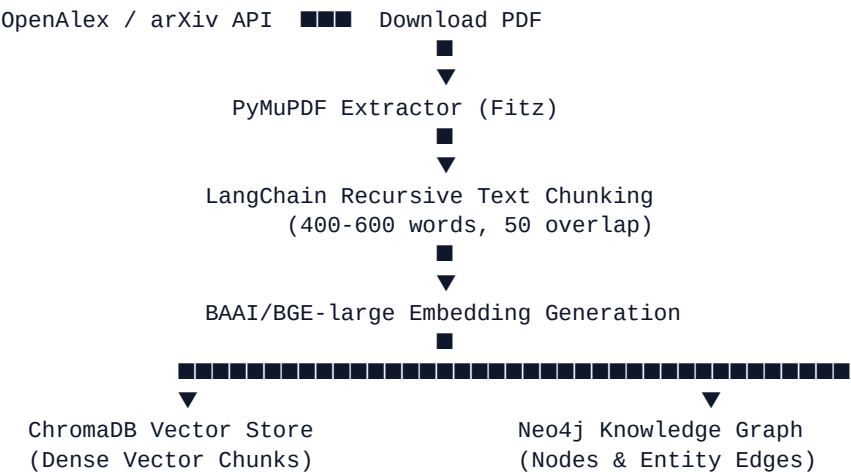
6.2 Technology Stack Summary

Layer / Module	Selected Technology	Role & Rationale
Frontend UI	React + TypeScript + Tailwind	SPA with Virtual DOM speed, type safety, and utility-first styling.
Backend API	FastAPI (Python)	High-speed asynchronous Python web framework with auto OpenAPI docs.
Paper Harvesting	OpenAlex API & arXiv API	Access to millions of open-access research metadata and PDFs.
PDF Parser	PyMuPDF (Fitz)	Ultra-fast text & metadata extraction from complex double-column academic PDFs.
Text Chunking	LangChain Recursive Text Splitter	Splits text into 400-600 word chunks with overlapping context bounds.
Embedding Model	BAAI/BGE-large-en-v1.5	Top-ranking open embedding model producing 1024-dim dense vectors.
Vector Database	ChromaDB	Lightweight, highly performant vector store optimized for cosine similarity.
Knowledge Graph	Neo4j Graph Database	Industry-standard property graph database mapping nodes and relationships.
AI Framework	LangChain Framework	Orchestrates vector search, Cypher queries, prompt templates, and LLMs.
LLM Service	Gemini / Llama 3	Generates natural language summaries, comparisons, and citation answers.

6.3 Detailed Technology Rationale

- **Why React & TypeScript?** React's Virtual DOM ensures instant visual updates during graph interactions, while TypeScript enforces strict data typing between API payloads and UI components.
- **Why FastAPI?** Built on Starlette and Pydantic, FastAPI provides standard Python async execution, vital for concurrent AI model calls, vector lookups, and graph queries.
- **Why Neo4j?** Native property graph representation allows executing multi-hop Cypher queries (e.g., finding which datasets are used by papers citing a baseline algorithm).
- **Why ChromaDB?** Provides high-throughput vector indexing and similarity search with zero cloud lock-in.
- **Why BAAI/BGE-large-en-v1.5?** Outperforms standard embeddings on MTEB benchmarks for scientific domain retrieval.

6.4 PDF Processing, Chunking & Embedding Pipeline



6.5 Knowledge Graph Node & Edge Structure

- **Nodes:** Paper, Author, Methodology, Dataset, Algorithm, ResearchDomain.
- **Relationships:** (Paper)-[:WRITTEN_BY]->(Author), (Paper)-[:USES_METHOD]->(Methodology), (Paper)-[:BENCHMARKED_ON]->(Dataset), (Paper)-[:CITES]->(Paper).

6.6 Why GraphRAG Instead of Traditional RAG?

Standard Retrieval-Augmented Generation (RAG) isolates document chunks based strictly on vector cosine similarity. While effective for localized context lookup, standard RAG fails when answering global research questions such as 'What are the trending datasets across all transformer papers published in 2024?' or 'Which methods overcome the speed limitation of paper X?'

ResearchGraph AI's GraphRAG Solution: *By executing dual-retrieval (vector search in ChromaDB + sub-graph traversal in Neo4j), the LLM receives both exact text passages and structured multi-hop relational paths. This hybrid context enables accurate literature reviews, verifiable citation generation, multi-paper comparison matrices, and automatic research gap discovery.*